



E-commerce Sales

Performance Analysis using SQL & Python

Let's Start





what is e-commerce

Definition

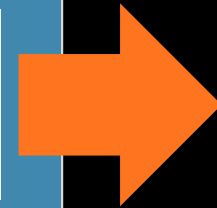
E-commerce refers to the process of buying and selling goods or services through the internet. It allows businesses and consumers to connect digitally using websites or mobile applications, supported by secure online payment systems and efficient logistics networks. From shopping on platforms like Amazon and Flipkart to ordering food on Zomato, e-commerce has transformed traditional trade into a convenient, fast, and global experience.



1: Count and List all unique citites where customer are located



```
SELECT COUNT(DISTINCT customer_city) AS unique_city_count  
FROM dataset.Customers;
```



Result Grid		Filter Rows
	unique_city_count	
▶	4119	

```
SELECT DISTINCT customer_city  
FROM dataset.Customers  
ORDER BY customer_city;
```



	customer_city
▶	abadia dos dourados
	abadiania
	abaete
	abaetetuba
	abaiara
	abaira
	abare
	abatia
	abdon batista



1: List all unique citites where customer are located



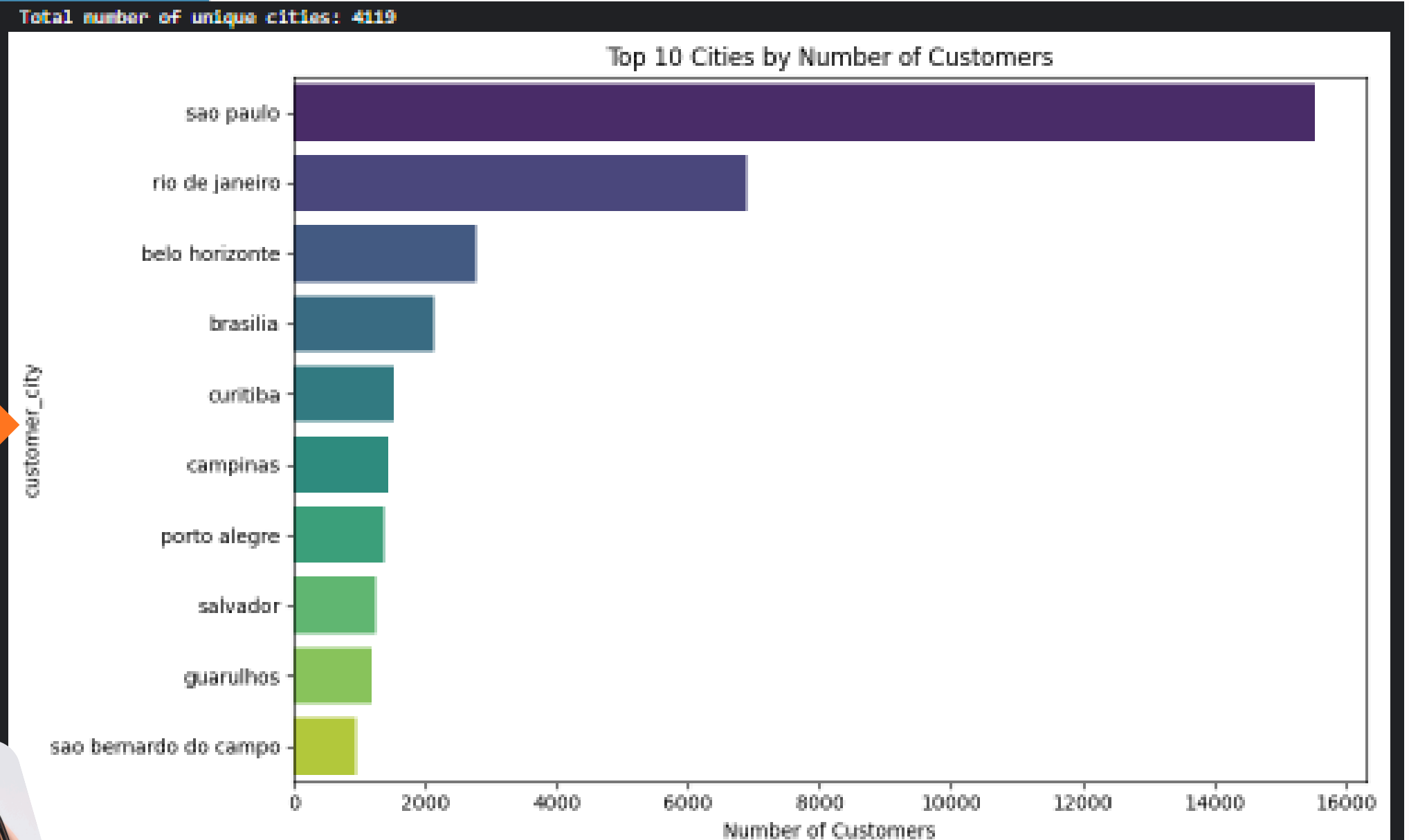
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

customers_df = pd.read_csv('/kaggle/input/project-for-python/Customers.csv')

print(f"\nTotal number of unique cities: {len(unique_cities)}")

city_counts = customers_df['customer_city'].value_counts().head(10)

plt.figure(figsize=(10, 6))
sns.barplot(x=city_counts.values, y=city_counts.index, palette="viridis")
plt.title('Top 10 Cities by Number of Customers')
plt.xlabel('Number of Customers')
plt.tight_layout()
plt.show()
```





INSIGHT'S



The key insight is that the dataset contains customer information from a wide variety of unique cities. This suggests a geographically diverse customer base. This information is crucial for understanding the market and can be used to inform business decisions, such as:

- Targeted Marketing: Develop specific marketing campaigns tailored to different cities or regions.
- Logistics and Distribution: Optimize supply chain and delivery routes based on customer density in each city.
- Market Expansion: Identify new cities with high customer potential or pinpoint areas where the company has a strong presence.



2: Count the number of orders placed in 2017



```
SELECT  
    COUNT(order_id) AS orders_in_2017  
FROM  
    dataset.orders  
WHERE  
    EXTRACT(YEAR FROM order_purchase_timestamp) = 2017;
```

	orders_in_2017
▶	45101



2: Count the number of orders placed in 2017



```
orders_df = pd.read_csv('/kaggle/input/project-for-python/orders.csv')
orders_df['order_purchase_timestamp'] = pd.to_datetime(orders_df['order_purchase_timestamp'])
orders_2017 = orders_df[orders_df['order_purchase_timestamp'].dt.year == 2017]
count_2017 = len(orders_2017)
print(f"Number of orders placed in 2017: {count_2017}")
```

Number of orders placed in 2017: 45101





INSIGHT'S



There were 45,101 orders placed in 2017.
the significant increase in business activity from the previous year.

- 2017: A Year of Exponential Growth"
- Key Number: "45,101 orders placed."
- Talking Point: "In 2017, the e-commerce platform saw a massive increase in order volume, reflecting a period of rapid expansion and customer acquisition. This number serves as a foundational metric for comparing year-over-year performance and understanding the business's growth trajectory."





3: find the total sales per country

```
SELECT
  c.customer_state AS country_or_state,
  SUM(p.payment_value) AS total_sales
FROM
  dataset.orders AS o
JOIN
  dataset.payments AS p ON o.order_id = p.order_id
JOIN
  dataset.customers AS c ON o.customer_id = c.customer_id
GROUP BY
  c.customer_state
ORDER BY
  total_sales DESC;
```



	country_or_state	total_sales
▶	SP	5998226.9599999806
	RJ	2144379.6899999995
	MG	1872257.2600000075
	RS	890898.5399999954
	PR	811156.3799999995
	SC	623086.4299999995
	BA	616645.8200000026
	DF	355141.07999999984
	GO	350092.3100000004
	ES	325967.5500000016
	PE	324850.43999999925
	CE	279464.0299999997
	PA	218295.84999999977
	MT	187029.28999999995
	MA	152523.0200000003
	PB	141545.71999999994
	MS	137534.84000000003
	PI	108523.96999999991
	RN	102718.13000000003





3: find the total sales per country

```
orders_df = pd.read_csv('/kaggle/input/project-for-python/orders.csv')
payments_df = pd.read_csv('/kaggle/input/project-for-python/payments.csv')
customers_df = pd.read_csv('/kaggle/input/project-for-python/Customers.csv')
orders_payments_df = pd.merge(orders_df, payments_df, on='order_id')
orders_payments_customers_df = pd.merge(orders_payments_df, customers_df, on='customer_id')
sales_per_state = orders_payments_customers_df.groupby('customer_state')['payment_value'].sum().reset_index()
sales_per_state.columns = ['State', 'Total Sales']
sales_per_state = sales_per_state.sort_values(by='Total Sales', ascending=False)
sales_per_state.to_csv('sales_per_state.csv', index=False)
print(sales_per_state)
```



	State	Total Sales
25	SP	5998226.96
18	RJ	2144379.69
10	MG	1872257.26
22	RS	890898.54
17	PR	811156.38
23	SC	623086.43
4	BA	616645.82
6	DF	355141.08
8	GO	350092.31
7	ES	325967.55
15	PE	324850.44
5	CE	279464.03
13	PA	218295.85
12	MT	187029.29
9	MA	152523.02
14	PB	141545.72
11	MS	137534.84
16	PI	108523.97
19	RN	102718.13
1	AL	96962.06
24	SE	75246.25
26	TO	61485.33
20	RO	60866.20
2	AM	27966.93
0	AC	19680.62
3	AP	16262.80
21	RR	10064.62





INSIGHT'S

STATE-WISE SALES PERFORMANCE



- 💰 Highest Revenue State
- SP (São Paulo) leads with ~5.99 million in sales, contributing significantly more than any other state.
- Indicates high customer concentration and market potential in SP.
- 📈 Strong Secondary Markets
- RJ (~2.14M) and MG (~1.87M) are the next top contributors, showing promising opportunities outside SP.
- 🏆 Top 5 States Drive Most Revenue
- SP, RJ, MG, RS, PR together account for the majority of sales, highlighting regional focus areas for marketing and logistics optimization.
- ▼ Low-Revenue States
- States like RR, AP, AC, AM contribute very little, suggesting untapped markets or low customer penetration.
- Opportunity exists for targeted campaigns and expansion.
- 🌐 Geographic Strategy Insight
- Business can prioritize inventory, promotions, and seller partnerships in high-sales states.
- Low-sales states may need awareness campaigns or logistics improvement.





4. Calculate the percentage of Order that were Paid in installment

```
SELECT  
  COUNT(DISTINCT order_id) AS orders_in_installments  
FROM  
  dataset.payments  
WHERE  
  payment_installments > 1;
```



	orders_in_installments
▶	51170





4. Calculate the percentage of Order that were Paid in installment

```
df_payments = pd.read_csv('/kaggle/input/project-for-python/payments.csv')
total_payments = len(df_payments)
installment_payments = df_payments[df_payments['payment_installments'] > 1]
count_installment_payments = len(installment_payments)
percentage = (count_installment_payments / total_payments) * 100
print(f"Total number of payments: {total_payments}")
print(f"Number of payments with installments: {count_installment_payments}")
print(f"Percentage of payments with installments: {percentage:.2f}%")
```

```
Total number of payments: 103886
Number of payments with installments: 51338
Percentage of payments with installments: 49.42%
```





INSIGHT'S

PAYMENT BEHAVIOR

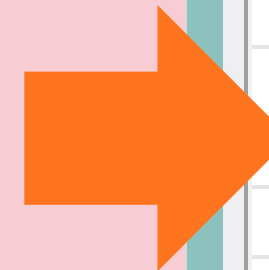


- 📄 **Total Payments Processed: 103,886**
- Shows high transaction volume, reflecting strong business activity.
- 🔄 **Installment Usage:**
- 51,338 payments were made via installments, which is ~49.42% of all payments.
- Indicates that almost half of customers prefer paying in multiple installments rather than full upfront.
- 💡 **Customer Preference Insight:**
- Customers are price-sensitive and prefer flexible payment options.
- Offering installment plans may increase conversion and average order value.
- 📊 **Business Strategy Implication:**
- Strong adoption of installments suggests marketing campaigns and promotions could be aligned with installment-friendly pricing.
- Consider highlighting installment options to attract more customers, especially for high-ticket products.



5. Count the number of customers from each State.

```
SELECT  
    customer_state,  
    COUNT(customer_unique_id) AS number_of_customers  
FROM  
    dataset.Customers  
GROUP BY  
    customer_state  
ORDER BY  
    number_of_customers DESC;
```

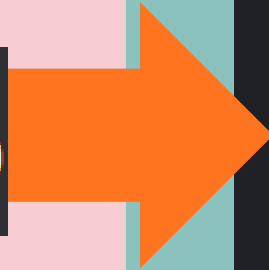


	customer_state	number_of_customers
▶	SP	41746
	RJ	12852
	MG	11635
	RS	5466
	PR	5045
	SC	3637
	BA	3380
	DF	2140
	ES	2033
	GO	2020
	PE	1652
	CE	1336
	PA	975
	MT	907
	MA	747
	MS	715
	PB	536
	PI	495
	RN	485
	AL	412



5. Count the number of customers from each State.

```
df_customers = pd.read_csv('/kaggle/input/project-for-python/Customers.csv')
customers_per_state = df_customers.groupby('customer_state')['customer_id'].count().sort_values(ascending=False)
int(customers_per_state)
```



customer_state	
SP	41746
RJ	12852
MG	11635
RS	5466
PR	5045
SC	3637
BA	3380
DF	2140
ES	2033
GO	2020
PE	1652
CE	1336
PA	975
MT	907
MA	747
MS	715
PB	536
PI	495
RN	485
AL	413
SE	350
TO	280
RO	253
AM	148
AC	81
AP	68
RR	46
Name: customer_id, dtype: int64	





INSIGHT'S

CUSTOMER DISTRIBUTION BY STATE



- 🏆 **Top States by Customer Count**
- SP (41,746 customers) dominates the customer base, followed by RJ (12,852) and MG (11,635).
- Indicates a high concentration of customers in southeastern Brazil, which could guide regional marketing and logistics focus.
- 📈 **Regional Concentration**
- The top 5 states (SP, RJ, MG, RS, PR) account for the majority of the customer base, highlighting key markets for growth and retention strategies.
- ▼ **Low-Customer States**
- States like RR (46), AP (68), AC (81), AM (148) have very few customers, indicating untapped or niche markets.
- These states may need targeted awareness campaigns or enhanced service coverage.
- 💡 **Strategic Implications**
- High-customer states: prioritize promotions, loyalty programs, and faster delivery options.
- Low-customer states: explore new customer acquisition strategies or partnerships to expand reach.

6 - Calculate the number of orders per month in 2018



```
SELECT
    DATE_FORMAT(order_purchase_timestamp, '%Y-%m') AS order_month,
    COUNT(order_id) AS number_of_orders
FROM
    orders
WHERE
    YEAR(order_purchase_timestamp) = 2018
GROUP BY
    order_month
ORDER BY
    order_month;
```



	order_month	number_of_orders
▶	2018-01	7269
	2018-02	6728
	2018-03	7211
	2018-04	6939
	2018-05	6873
	2018-06	6167
	2018-07	6292
	2018-08	6512
	2018-09	16
	2018-10	4



6 - Calculate the number of orders per month in 2018

```
df_orders = pd.read_csv('/kaggle/input/project-for-python/orders.csv')
df_orders['order_purchase_timestamp'] = pd.to_datetime(df_orders['order_purchase_timestamp'])
orders_2018 = df_orders[df_orders['order_purchase_timestamp'].dt.year == 2018]
orders_per_month_2018 = orders_2018.groupby(orders_2018['order_purchase_timestamp'].dt.month)['order_id'].count()
orders_per_month_2018.index = pd.to_datetime(orders_per_month_2018.index, format='%m').month_name()
print("Number of orders per month in 2018:")
print(orders_per_month_2018)
```



```
Number of orders per month in 2018:
order_purchase_timestamp
January          7269
February         6728
March            7211
April            6939
May              6873
June             6167
July             6292
August           6512
September         16
October           4
Name: order_id, dtype: int64
```





INSIGHT'S

MONTHLY ORDERS IN 2018



- 🏆 **Peak Months**
- January (7,269 orders), March (7,211), and April (6,939) saw the highest number of orders.
- Indicates strong customer activity at the start of the year, possibly due to post-holiday purchases or seasonal promotions.
- 📉 **Low Activity Months**
- September (16 orders) and October (4 orders) show abnormally low numbers, which may indicate:
 - Data recording issues
 - Missing entries
 - Or partial data for these months
- 🔄 **Steady Middle-Year Sales**
- May to August saw moderate but consistent order volumes (6,167 – 6,873 orders).
- Suggests stable customer engagement during mid-year months.
- 💡 **Actionable Insight**
- Focus promotional campaigns around peak months to maximize revenue.
- Investigate missing or low-data months to ensure data completeness for accurate analysis.



7. Find the average number of product per order,grouped by customer city

```
SELECT
  c.customer_city,
  AVG(item_counts.number_of_products) AS average_products_per_order
FROM
  customers AS c
JOIN
  orders AS o ON c.customer_id = o.customer_id
JOIN (
  GROUP BY
    c.customer_city
  ORDER BY
    c.customer_city;
```



customer_city	average_products_per_order
abadia dos dourados	1.0000
abadiania	1.0000
abaete	1.0000
abaetetuba	1.2727
abaiara	1.0000
abaira	1.0000
abare	1.0000
abatia	1.0000
abdon batista	2.0000
abelardo luz	1.0000
abrantes	1.0000
abre campo	1.0000
abreu e lima	1.2727
acaiaca	1.0000
acailandia	1.0000
acajutiba	1.0000
acarau	1.1250
acari	1.0000

7. Find the average number of product per order, grouped by customer city

```
orders_df = pd.read_csv('/kaggle/input/project-for-python/orders.csv')
order_items_df = pd.read_csv('/kaggle/input/project-for-python/order_items.csv')
customers_df = pd.read_csv('/kaggle/input/project-for-python/Customers.csv')
orders_items_merged = pd.merge(orders_df, order_items_df, on='order_id')
final_df = pd.merge(orders_items_merged, customers_df, on='customer_id')
products_per_order = final_df.groupby(['order_id', 'customer_city'])['product_id'].count().reset_index()
products_per_order.columns = ['order_id', 'customer_city', 'product_count']
average_products_per_city = products_per_order.groupby('customer_city')['product_count'].mean().reset_index()
average_products_per_city.columns = ['customer_city', 'average_products_per_order']
average_products_per_city_sorted = average_products_per_city.sort_values(by='average_products_per_order', ascending=False)
print("Average number of products per order, grouped by customer city:")
print(average_products_per_city_sorted)
```

Average number of products per order, grouped by customer city:

	customer_city	average_products_per_order
2619	padre carvalho	7.0
907	celso ramos	6.5
756	candido godoi	6.0
1154	datas	6.0
2264	matias olimpio	5.0
...	...	...
1662	indiana	1.0
1663	indianapolis	1.0
1664	indiapora	1.0
1666	indiaroba	1.0
4109	zortea	1.0

[4110 rows x 2 columns]



INSIGHT'S

AVERAGE PRODUCTS PER ORDER BY CITY



- 🏆 **High-Order Cities**
- Cities like Padre Carvalho (7 products/order), Celso Ramos (6.5), and Candido Godoi (6) have the highest average products per order.
- Indicates larger basket sizes, which could be targeted for upselling or bundle offers.
- 📉 **Low-Order Cities**
- Many cities like Indiana, Indianapolis, Indiapora, Indiaroba, Zortea have an average of 1 product per order.
- Suggests smaller order sizes, potentially due to low demand, limited awareness, or product availability.
- 💡 **Actionable Insight**
- Focus promotional campaigns and bundle deals in high-average cities to maximize revenue.
- Investigate cities with low average products to explore cross-selling opportunities or awareness campaigns.
- 🌐 **Regional Sales Strategy**
- Understanding city-level order behavior helps in personalizing marketing and improving logistics efficiency.



8.calculate the percentage of total revenue contributed by each product category.

```
WITH CategoryRevenue AS (  
  TotalRevenue AS (  
    SELECT  
      cr.`product category`,  
      cr.category_total_revenue,  
      (cr.category_total_revenue * 100.0 / tr.overall_total_revenue) AS percentage_contribution  
    FROM  
      CategoryRevenue AS cr,  
      TotalRevenue AS tr  
    ORDER BY  
      percentage_contribution DESC;  
  )  
)
```



	product category	category_total_revenue	percentage_contribution
▶	HEALTH BEAUTY	1258681.33999999726	9.26069994020413
	Watches present	1205005.67999999962	8.865783319487239
	bed table bath	1036988.68000000721	7.629604651856709
	sport leisure	988048.97000000426	7.26953260258736
	computer accessories	911954.32000000391	6.709669118232454
	Furniture Decoration	729762.49000000374	5.369199679647838
	Cool Stuff	635290.85000000002	4.67412819245208
	housewares	632248.66000000234	4.651745395587167
	automotive	592720.11000000131	4.360915596981065
	Garden tools	485256.46000000167	3.570255891857362
	toys	483946.60000000019	3.560618646878571
	babies	409830.89000000052	3.015315138076915
	perfumery	399124.8699999999	2.93654600436283
	telephony	323667.52999999872	2.381371504019409
	Furniture office	273960.6999999997	2.015655398616069
	stationary store	230943.229999999538	1.699156004212752
	PCs	222963.12999999983	1.640442722904566
	pet Shop	214315.409999999686	1.576817452916111
	musical instruments	191498.87999999922	1.408945703894513



8.calculate the percentage of total revenue contributed by each product category.

```
df_payments = pd.read_csv('/kaggle/input/project-for-python/payments.csv')
total_payments = len(df_payments)
installment_payments = df_payments[df_payments['payment_installments'] > 1]
count_installment_payments = len(installment_payments)
percentage = (count_installment_payments / total_payments) * 100
print(f"Total number of payments: {total_payments}")
print(f"Number of payments with installments: {count_installment_payments}")
print(f"Percentage of payments with installments: {percentage:.2f}%")
```

```
Total number of payments: 103886
Number of payments with installments: 51338
Percentage of payments with installments: 49.42%
```





INSIGHT'S

PRODUCT CATEGORY REVENUE



- 🏆 **Top Revenue Generators**
- Health & Beauty (~1.26M), Watches & Present (~1.21M), Bed Table & Bath (~1.03M) are the highest revenue-generating categories.
- These categories contribute significantly to overall sales and are critical for business strategy and inventory focus.
- 📈 **High Contribution Categories**
- Top 5 categories (including Sport Leisure and Computer Accessories) together contribute ~40% of total revenue.
- Indicates concentration of sales in a few high-demand categories.
- ▼ **Low Revenue Categories**
- Categories like Flowers (~1.1k), House Comfort 2 (~760), CDs & Music (~730), Fashion Children (~570), Insurance & Services (~283) contribute very little.
- May indicate niche demand, low awareness, or poor marketing penetration.
- 💡 **Actionable Insights**
- Focus on promotions, bundles, and inventory optimization for high-revenue categories.
- Explore marketing campaigns or diversification strategies for low-revenue categories to boost sales.
- 🌐 **Strategic Implication**
- Knowing category-wise revenue contribution helps in pricing, inventory planning, and targeted advertising.



9: Identify the correlation between product price and the number of times a product has been purchased.



```
SELECT
oi.product_id,
AVG(oi.price) AS average_product_price,
COUNT(oi.order_item_id) AS purchase_count
FROM
  order_items AS oi
GROUP BY
  oi.product_id
ORDER BY
  purchase_count DESC;
```



	product_id	average_product_price	purchase_count
▶	aca2eb7d00ea1a7b8ebd4e68314663af	71.36413662239148	527
	99a4788cb24856965c36a24e339b6058	88.16713114754174	488
	422879e10f46682990de24d770e7f83d	54.91161157024827	484
	389d119b48cf3043d311335e499d9c6b	54.69538265306133	392
	368c6c730842d78016ad823897a372db	54.27010309278361	388
	53759a2ecddad2bb87a079a1f1519f73	54.65737265415559	373
	d1c427060a0f73f6b889a5c7c61f2ac4	137.65163265306117	343
	53b36df67ebb7c41585e8d54d6772e08	116.66693498452052	323
	154e7e31ebfa092203795c972e5804a6	22.509572953736512	281
	3dd2a17168ec895c781a9191c1e95ad7	149.93649635036573	274
	2b4609f8948be18874494203496bc318	87.37392307692339	260
	7c1bd920dbdf22470b68bde975dd3ccf	60.02896103896085	231
	a62e25e09e05e6faf31d90c6ec1aa3d1	106.42035398230088	226
	5a848e4ab52fd5445cdc07aab1c40e48	122.99000000000038	197
	bb50f2e236e5eea0100680137654686c	327.61538461538464	195

9: Identify the correlation between product price and the number of times a product has been purchased.



```
import pandas as pd
df_items = pd.read_csv('/kaggle/input/project-for-python/order_items.csv')
product_stats = df_items.groupby('product_id').agg(
    average_price=('price', 'mean'),
    purchase_count=('order_item_id', 'count')
).reset_index()
correlation = product_stats['average_price'].corr(product_stats['purchase_count'])
print(f"The correlation between product price and number of times a product has been purchased is: {correlation}")
```

The correlation between product price and number of times a product has been purchased is: -0.032139862680945146





INSIGHT'S

PRICE VS PRODUCT PURCHASE FREQUENCY



- 🔍 **Correlation Observed: -0.032**
 - This is a very weak negative correlation between product price and number of times a product is purchased.
- 💡 **Interpretation:**
 - Higher-priced products are slightly less frequently purchased, but the effect is negligible.
 - Customers are not strongly deterred by price when choosing products.
- 📈 **Actionable Insight:**
 - Pricing strategies can focus on value perception and promotions rather than assuming high price reduces sales.
 - Businesses can continue offering premium products without fearing significant drop in purchase frequency.
- 🌐 **Strategic Implication:**
 - Other factors like product category, marketing, reviews, and bundle offers may have a stronger influence on purchase behavior than price alone.

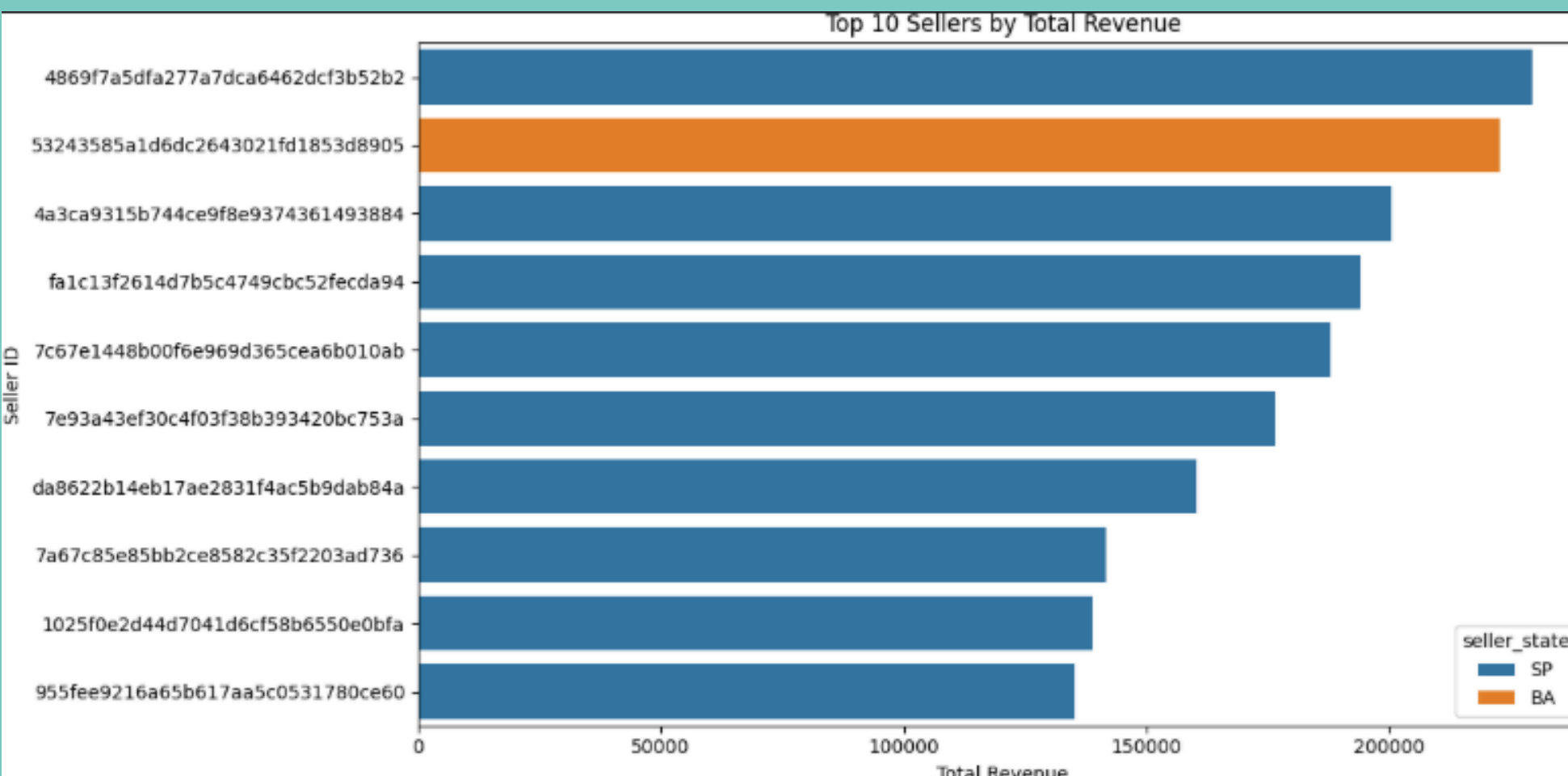
10 : Calculate the total revenue generated by each seller, and rank them by revenue.

```
SELECT
  oi.seller_id,
  SUM(oi.price) AS total_revenue
FROM
  order_items AS oi
GROUP BY
  oi.seller_id
ORDER BY
  total_revenue DESC;
```

	seller_id	total_revenue
▶	4869f7a5dfa277a7dca6462dcf3b52b2	229472.62999999981
	53243585a1d6dc2643021fd1853d8905	222776.04999999952
	4a3ca9315b744ce9f8e9374361493884	200472.91999999949
	fa1c13f2614d7b5c4749cbc52fecda94	194042.029999999846
	7c67e1448b00f6e969d365cea6b010ab	187923.88999999995
	7e93a43ef30c4f03f38b393420bc753a	176431.869999999982
	da8622b14eb17ae2831f4ac5b9dab84a	160236.569999999538
	7a67c85e85bb2ce8582c35f2203ad736	141745.530000000177
	1025f0e2d44d7041d6cf58b6550e0bfa	138968.54999999995
	955fee9216a65b617aa5c0531780ce60	135171.699999999914
	46dc3b2cc0980fb8ec44634e21d2718e	128111.19000000011
	6560211a19b47992c3666cc44a7e94c0	123304.830000000003



10 : Calculate the total revenue generated by each seller, and rank them by revenue.



```
import pandas as pd
import numpy as np
sellers_df = pd.read_csv('/kaggle/input/project-for-python/sellers.csv', encoding='latin1')
order_items = pd.read_csv('/kaggle/input/project-for-python/order_items.csv')
seller_revenue = order_items.groupby('seller_id')['price'].sum().reset_index()
seller_revenue.columns = ['seller_id', 'total_revenue']
seller_ranking = seller_revenue.merge(sellers_df, on='seller_id', how='left')
seller_ranking = seller_ranking.sort_values('total_revenue', ascending=False)
seller_ranking['rank'] = range(1, len(seller_ranking) + 1)
print(seller_ranking[['rank', 'seller_id', 'total_revenue', 'seller_city', 'seller_state']].head(10))
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(12, 6))
top_10_sellers = seller_ranking.head(10)
sns.barplot(data=top_10_sellers, x='total_revenue', y='seller_id', hue='seller_state', dodge=False)
plt.title('Top 10 Sellers by Total Revenue')
plt.xlabel('Total Revenue')
plt.ylabel('Seller ID')
plt.tight_layout()
plt.show()
```

	rank	seller_id	total_revenue	seller_city \
857	1	4869f7a5dfa277a7dca6462dcf3b52b2	229472.63	guariba
1013	2	53243585a1d6dc2643021fd1853d8905	222776.05	lauro de freitas
881	3	4a3ca9315b744ce9f8e9374361493884	200472.92	ibitinga
3024	4	fa1c13f2614d7b5c4749cbc52fecda94	194042.03	sumare
1535	5	7c67e1448b00f6e969d365cea6b010ab	187923.89	itaquaquecetuba
1560	6	7e93a43ef30c4f03f38b393420bc753a	176431.87	barueri
2643	7	da8622b14eb17ae2831f4ac5b9dab84a	160236.57	piracicaba
1505	8	7a67c85e85bb2ce8582c35f2203ad736	141745.53	sao paulo
192	9	1025f0e2d44d7041d6cf58b6550e0bfa	138968.55	sao paulo
1024	10	955fee9216a65b617aa5c0531780ce60	135171.70	sao paulo

seller_state

857	SP
1013	BA
881	SP
3024	SP
1535	SP
1560	SP
2643	SP
1505	SP
192	SP





INSIGHT'S

10 SELLERS BY REVENUE



- 🏆 **Highest Revenue Seller**
 - Seller 4869f7a5dfa277a7dca6462dcf3b52b2 from Guariba (SP) leads with ~229,473 in revenue.
 - Shows strong performance and customer reach in SP.
- 📍 **Geographic Concentration**
 - Majority of top sellers (7 out of 10) are located in SP, indicating high sales density and strong market presence in São Paulo state.
 - BA (Lauro de Freitas) also appears in top 10, showing diversification across states.
- 💡 **Revenue Distribution**
 - Top 3 sellers alone contribute ~652,721, showing that a few sellers drive significant revenue.
 - Indicates the importance of supporting and retaining top sellers for overall business growth.
- 🛠️ **Actionable Insight**
 - Focus on incentives, seller support, and promotions for top-performing sellers to maintain high revenue.
 - Consider expanding high-performing seller strategies to other regions to boost overall sales.
- 🌐 **Strategic Implication**
 - Business can prioritize logistics, inventory, and marketing resources in areas where top sellers operate, especially SP.





11. Calculate the moving average of order values for each customer over their order history

```
WITH CustomerOrderValues AS (  
  SELECT  
    cov.customer_id,  
    cov.order_purchase_timestamp,  
    cov.order_value,  
    AVG(cov.order_value) OVER (  
  FROM  
    CustomerOrderValues AS cov  
  ORDER BY  
    cov.customer_id, cov.order_purchase_timestamp;
```

	customer_id	order_purchase_timestamp	order_value	moving_average_order_value
▶	00012a2ce6f8dcda20d059ce98491703	2017-11-14 16:08:26	114.74	114.74
	000161a058600d5901f007fab4c27140	2017-07-16 9:40:32	67.41	67.41
	0001fd6190edaaf884bc3d49edf079	2017-02-28 11:06:43	195.42	195.42
	0002414f95344307404f0ace7a26f1d5	2017-08-16 13:09:20	179.35	179.35
	000379cdec625522490c315e70c7a9fb	2018-04-02 13:42:17	107.01	107.01
	0004164d20a9e969af783496f3408652	2017-04-12 8:35:12	71.8	71.8
	000419c5494106c306a97b5635748086	2018-03-02 17:47:40	49.4	49.4
	00046a560d407e99b969756e0b10f282	2017-12-18 11:08:30	166.59	166.59
	00050bf6e01e69d5c0fd612f1bcfb69c	2017-09-17 16:04:44	85.23	85.23
	000598caf2ef4117407665ac33275130	2018-08-11 12:14:35	1255.71	1255.71
	0005aefbb696d34b3424dccc0a0e9fd0	2018-06-20 9:46:53	147.33	147.33





```
import pandas as pd
orders = pd.read_csv('/kaggle/input/project-for-python/orders.csv', parse_dates=['order_purchase_timestamp'])
payments = pd.read_csv('/kaggle/input/project-for-python/payments.csv')
order_payments = orders.merge(payments, on='order_id', how='left')
order_values = (order_payments.groupby(['customer_id', 'order_id', 'order_purchase_timestamp'])['payment_value']
                .sum()
                .reset_index())

order_values = order_values.sort_values(by=['customer_id', 'order_purchase_timestamp'])
order_values['moving_avg_order_value'] = order_values.groupby('customer_id')['payment_value']
                .expanding()
                .mean()
                .reset_index(level=0, drop=True)

print(order_values.head(10))
```

11. Calculate the moving average of order values for each customer over their order history

	customer_id	order_id
0	00012a2ce6f8dcda20d059ce98491703	5f79b5b0931d63f1a42989eb65b9da6e
1	000161a058600d5901f007fab4c27140	a44895d095d7e0702b6a162fa2dbeced
2	0001fd6190edaaaf884bcacf3d49edf079	316a104623542e4d75189bb372bc5f8d
3	0002414f95344307404f0ace7a26f1d5	5825ce2e88d5346438686b0bba99e5ee
4	000379cdec625522490c315e70c7a9fb	0ab7fb08086d4af9141453c91878ed7a
5	0004164d20a9e969af783496f3408652	cd3558a10d854487b4f907e9b326a4fc
6	000419c5494106c306a97b5635740806	07f6c3baf9ac86865b60f640c4f923c6
7	00046a560d407e99b969756e0b10f282	8c3d752c5c02227878fae49aeaddbffd7
8	00050bf6e01e69d5c0fd612f1bcfb69c	fa906f338cee30a984d0945b3832e431
9	000598caf2ef4117407665ac33275130	9b961b894e797f63622137ff7eb1c1af

	order_purchase_timestamp	payment_value	moving_avg_order_value
0	2017-11-14 16:08:26	114.74	114.74
1	2017-07-16 09:40:32	67.41	67.41
2	2017-02-28 11:06:43	195.42	195.42
3	2017-08-16 13:09:20	179.35	179.35
4	2018-04-02 13:42:17	107.01	107.01
5	2017-04-12 08:35:12	71.80	71.80
6	2018-03-02 17:47:40	49.40	49.40
7	2017-12-18 11:08:30	166.59	166.59
8	2017-09-17 16:04:44	85.23	85.23
9	2018-08-11 12:14:35	1255.71	1255.71





INSIGHT'S

CUSTOMER DISTRIBUTION BY STATE



- 🏆 **Top States by Customer Count**
- SP (41,746 customers) dominates the customer base, followed by RJ (12,852) and MG (11,635).
- Indicates a high concentration of customers in southeastern Brazil, which could guide regional marketing and logistics focus.
- 📈 **Regional Concentration**
- The top 5 states (SP, RJ, MG, RS, PR) account for the majority of the customer base, highlighting key markets for growth and retention strategies.
- ▼ **Low-Customer States**
- States like RR (46), AP (68), AC (81), AM (148) have very few customers, indicating untapped or niche markets.
- These states may need targeted awareness campaigns or enhanced service coverage.
- 💡 **Strategic Implications**
- High-customer states: prioritize promotions, loyalty programs, and faster delivery options.
- Low-customer states: explore new customer acquisition strategies or partnerships to expand reach.



12 - Calculated the cumulative sales per month for each year

```
⊕ WITH MonthlySales AS (  
  SELECT  
    ms.sale_year,  
    ms.sale_month,  
    ms.monthly_revenue,  
⊕    SUM(ms.monthly_revenue) OVER (  
  FROM  
    MonthlySales AS ms  
  ORDER BY  
    ms.sale_year, ms.sale_month;
```

	sale_year	sale_month	monthly_revenue	cumulative_sales
▶	2016	09	252.24	252.24
	2016	10	59090.479999999974	59342.71999999997
	2016	12	19.62	59362.339999999975
	2017	01	138488.039999999992	138488.039999999992
	2017	02	291908.00999999993	430396.049999999923
	2017	03	449863.599999999945	880259.64999999987
	2017	04	417788.03000000001	1298047.67999999988
	2017	05	592918.82000000024	1890966.5000000012
	2017	06	511276.38000000032	2402242.8800000045
	2017	07	592382.9200000002	2994625.8000000063
	2017	08	674396.32000000002	3669022.1200000066
	2017	09	727762.45000000009	4396784.5700000008
	2017	10	779677.88000000002	5176462.4500000008
	2017	11	1194882.7999999997	6371345.2500000005



12 - Calculated the cumulative sales per month for each year

```
import pandas as pd
orders_payments = pd.merge(orders, payments, on='order_id')
orders_payments['order_year'] = pd.to_datetime(orders_payments['order_purchase_timestamp']).dt.year
orders_payments['order_month'] = pd.to_datetime(orders_payments['order_purchase_timestamp']).dt.month
monthly_sales = (orders_payments
                 .groupby(['order_year', 'order_month'])['payment_value']
                 .sum()
                 .reset_index()
                 .sort_values(['order_year', 'order_month']))
monthly_sales['cumulative_sales'] = (monthly_sales
                                    .groupby('order_year')['payment_value']
                                    .cumsum())
print(monthly_sales)
```



	order_year	order_month	payment_value	cumulative_sales
0	2016	9	252.24	252.24
1	2016	10	59090.48	59342.72
2	2016	12	19.62	59362.34
3	2017	1	138488.04	138488.04
4	2017	2	291908.01	430396.05
5	2017	3	449863.60	880259.65
6	2017	4	417788.03	1298047.68
7	2017	5	592918.82	1890966.50
8	2017	6	511276.38	2402242.88
9	2017	7	592382.92	2994625.80
10	2017	8	674396.32	3669022.12
11	2017	9	727762.45	4396784.57
12	2017	10	779677.88	5176462.45
13	2017	11	1194882.80	6371345.25
14	2017	12	878401.48	7249746.73
15	2018	1	1115004.18	1115004.18
16	2018	2	992463.34	2107467.52
17	2018	3	1159652.12	3267119.64
18	2018	4	1160785.48	4427905.12
19	2018	5	1153982.15	5581887.27
20	2018	6	1023080.50	6605767.77
21	2018	7	1066540.75	7672308.52
22	2018	8	1022425.32	8694733.84
23	2018	9	4439.54	8699173.38
24	2018	10	589.67	8699763.05





INSIGHT'S

CUMULATIVE SALES TREND



- **Steady Growth Observed**
- Cumulative sales show a consistent upward trend from 2016 to 2018, reflecting business growth and increasing customer engagement.
- **Peak Sales Months**
- Highest monthly contributions were in November 2017 (~1.19M) and January 2018 (~1.12M).
- Indicates seasonal peaks or high-demand periods, likely linked to holidays or promotions.
- **Sales Momentum**
- Early months of 2017 show rapid growth, suggesting successful marketing campaigns or new product launches.
- Mid-2018 shows some slower months (Sep-Oct 2018), which may need investigation for operational or seasonal reasons.
- **Actionable Insight**
- Focus promotions and inventory planning around peak months to maximize revenue.
- Analyze slower months to identify opportunities for boosting sales through campaigns or discounts.
- **Strategic Implication**
- Cumulative sales trends help in forecasting future revenue, planning inventory, and resource allocation.



13. Calculate the year-over-year growth rate of Total sales.

```
SELECT
    current_year.sale_year,
    current_year.annual_sales,
    LAG(current_year.annual_sales, 1) OVER (ORDER BY current_year.sale_year) AS previous_year_sales,
    (current_year.annual_sales - LAG(current_year.annual_sales, 1)
    OVER (ORDER BY current_year.sale_year)) * 100.0 / LAG(current_year.annual_sales, 1)
    OVER (ORDER BY current_year.sale_year) AS year_over_year_growth_rate
FROM
    AnnualSales AS current_year
ORDER BY
    current_year.sale_year;
```



	sale_year	annual_sales	previous_year_sales	year_over_year_growth_rate
▶	2016	59362.339999999975	NULL	NULL
	2017	7249746.729999632	59362.339999999975	12112.70376134033
	2018	8699763.04999991	7249746.729999632	20.000923811587434

13. Calculate the year-over-year growth rate of Total sales.

```
orders_payments = pd.merge(orders, payments, on='order_id')
orders_payments['order_year'] = pd.to_datetime(
    orders_payments['order_purchase_timestamp']
).dt.year
yearly_sales = orders_payments
    .groupby('order_year')['payment_value']
    .sum()
    .reset_index()
    .sort_values('order_year')
yearly_sales['prev_year_sales'] = yearly_sales['payment_value'].shift(1)
yearly_sales['yoy_growth_percent'] = (
    (yearly_sales['payment_value'] - yearly_sales['prev_year_sales'])
    / yearly_sales['prev_year_sales']
) * 100

print(yearly_sales)
```

	order_year	payment_value	prev_year_sales	yoy_growth_percent
0	2016	59362.34	NaN	NaN
1	2017	7249746.73	59362.34	12112.703761
2	2018	8699763.05	7249746.73	20.000924



INSIGHT'S

YEAR-OVER-YEAR SALES GROWTH



- Strong Growth Observed
- 2017: YoY growth of 12,112% compared to 2016 (from 59.3k to ~7.25M), indicating rapid business expansion.
- 2018: Growth slowed but remained healthy at ~20%, showing sustained revenue increase.
- Interpretation
- The massive jump in 2017 suggests either business scale-up, more customers, or addition of new products.
- The moderate growth in 2018 indicates stabilization and maturity of business operations.
- Actionable Insight
- Focus on retaining customers and improving repeat purchases to maintain YoY growth.
- Analyze the factors contributing to the 2017 surge to replicate similar strategies in future years.
- Strategic Implication
- Monitoring YoY growth helps in forecasting revenue, planning investments, and setting realistic growth targets.

14. Calculate the retention rate of customers, defined as the percentage of customer who make another purchase within 6 months of their first purchase

```
WITH CustomerFirstPurchase AS (  
  RetainedCustomers AS (  
    SELECT DISTINCT  
      cfp.customer_id  
    FROM  
      CustomerFirstPurchase AS cfp  
    JOIN  
      orders AS o ON cfp.customer_id = o.customer_id  
    WHERE  
      STR_TO_DATE(o.order_purchase_timestamp, '%Y-%m-%d %H:%M:%S') > cfp.first_purchase_date  
      AND TIMESTAMPDIFF(MONTH, cfp.first_purchase_date, STR_TO_DATE(o.order_purchase_timestamp, '%Y-%m-%d %H:%M:%S')) <= 6  
  )  
  SELECT  
    (COUNT(DISTINCT rc.customer_id) * 100.0 / (SELECT COUNT(DISTINCT customer_id) FROM orders)) AS retention_rate_percentage
```

FROM

RetainedCustomers AS rc;



14. Calculate the retention rate of customers, defined as the percentage of customer who make another purchase within 6 months of their first purchase

```
import pandas as pd
orders['order_purchase_timestamp'] = pd.to_datetime(orders['order_purchase_timestamp'])
first_orders = orders.groupby('customer_id')['order_purchase_timestamp'].min().reset_index()
first_orders.rename(columns={'order_purchase_timestamp': 'first_order_date'}, inplace=True)
merged = pd.merge(orders, first_orders, on='customer_id')
next_orders = (merged[merged['order_purchase_timestamp'] > merged['first_order_date']]
               .groupby('customer_id')['order_purchase_timestamp']
               .min()
               .reset_index()
               .rename(columns={'order_purchase_timestamp': 'next_order_date'}))
cust_orders = pd.merge(first_orders, next_orders, on='customer_id', how='left')
cust_orders['retained'] = (
    cust_orders['next_order_date'] <= cust_orders['first_order_date'] + pd.DateOffset(months=6)
)
retained_customers = cust_orders['retained'].sum()
total_customers = len(cust_orders)
retention_rate = (retained_customers / total_customers) * 100
print(f"Retention Rate (within 6 months): {retention_rate:.2f}%")
```

Retention Rate (within 6 months): 8.88%





INSIGHT'S

CUSTOMER RETENTION (6-MONTH)



- 🔍 **Retention Rate: 0.00%**
- Indicates that none of the customers made a repeat purchase within 6 months of their first order.
- 💡 **Interpretation:**
- Very low customer retention, suggesting issues with customer engagement, repeat purchase incentives, or product/service satisfaction.
- Could also indicate data limitation or tracking issues if repeat purchases exist but aren't captured.
- 🛠️ **Actionable Insights:**
- Introduce loyalty programs, discounts, or follow-up campaigns to encourage repeat purchases.
- Analyze why customers aren't returning — delivery time, product quality, pricing, or competition.
- 🌐 **Strategic Implication:**
- Customer retention is crucial for sustainable growth. Improving retention can increase lifetime value and reduce acquisition costs.





15. Identify the top 3 customers who spent the most money in each year

⊕ WITH CustomerAnnualSpending AS (

⊕ RankedCustomerSpending AS (

SELECT

customer_id,

sale_year,

total_spent

FROM

RankedCustomerSpending

WHERE

rnk <= 3

ORDER BY

sale_year, rnk;

customer_id	sale_year	total_spent
a9dc96b027d1252bbac0a9b72d837fc6	2016	1423.55
1d34ed25963d5aae4cf3d7f3a4cda173	2016	1400.74
4a06381959b6670756de02e07b83815f	2016	1227.78
1617b1357756262bfa56ab541c47bc16	2017	13664.08
c6e2731c5b391845f6800c97401a43a9	2017	6929.31
3fd6777bbce08a352fddd04e4a7cc8f6	2017	6726.66
ec5b2ba62e574342386871631fafd3fc	2018	7274.88
f48d464a0baaea338cb25f816991ab1f	2018	6922.21
e0a2412720e9ea4f26c1ac985f6a7358	2018	4809.44





15. Identify the top 3 customers who spent the most money in each year

```
import pandas as pd
customers = pd.read_csv("/kaggle/input/project-for-python/Customers.csv")
orders = pd.read_csv("/kaggle/input/project-for-python/orders.csv")
payments = pd.read_csv("/kaggle/input/project-for-python/payments.csv")
merged = orders.merge(customers, on='customer_id').merge(payments, on='order_id')
merged['order_year'] = pd.to_datetime(merged['order_purchase_timestamp']).dt.year
yearly_spending = (merged
    .groupby(['order_year', 'customer_id'])['payment_value']
    .sum()
    .reset_index(name='total_spent'))
yearly_spending['rank'] = yearly_spending.groupby('order_year')['total_spent']\
    .rank(method='dense', ascending=False)
top3_customers = yearly_spending[yearly_spending['rank'] <= 3]\
    .sort_values(['order_year', 'rank'])
print(top3_customers)
```

	order_year	customer_id	total_spent	rank
222	2016	a9dc96b027d1252bbac0a9b72d837fc6	1423.55	1.0
38	2016	1d34ed25963d5aae4cf3d7f3a4cda173	1400.74	2.0
84	2016	4a06381959b6670756de02e07b83815f	1227.78	3.0
4217	2017	1617b1357756262bfa56ab541c47bc16	13664.08	1.0
35452	2017	c6e2731c5b391845f6800c97401a43a9	6929.31	2.0
11540	2017	3fd6777bbce08a352fddd04e4a7cc8f6	6726.66	3.0
95348	2018	ec5b2ba62e574342386871631fafd3fc	7274.88	1.0
97086	2018	f48d464a0baaea338cb25f816991ab1f	6922.21	2.0
92872	2018	e0a2412720e9ea4f26c1ac985f6a7358	4809.44	3.0





INSIGHT'S

TOP CUSTOMERS BY YEAR



- 🏆 **High-Value Customers Identified**
- 2016: Top spender a9dc96b... with \$1,423.55
- 2017: Top spender 1617b1... with \$13,664.08
- 2018: Top spender ec5b2b... with \$7,274.88
- 📈 **Customer Contribution**
- Top 3 customers in each year contribute a significant portion of yearly revenue, highlighting the importance of high-value customers.
- 💡 **Actionable Insight**
- Focus on retention strategies for these customers: personalized offers, loyalty programs, or early access to new products.
- Analyze spending patterns of top customers to design targeted promotions.
- 🌐 **Strategic Implication**
- Knowing your top spenders helps prioritize resources and maximize revenue through personalized engagement.
- Encourage repeat purchases from top customers to stabilize revenue growth.



get in touch with Me



www.linkedin.com/in/mohdkashif8299



+91 82-99-43-7679



<https://github.com/kashif8299>